

카메라Camera와 투영Projection

동명대학교 게임공학과

강영민

코드 분리 – myglfw.py

```
import glfw
#####
def initialize_window(w, h, window_title):
    if not glfw.init():
        return None
    window = glfw.create_window(w, h, window_title, None, None)
    if not window:
        glfw.terminate()
        print("window initialization failed")
        exit(0)

    glfw.make_context_current(window)
    return window

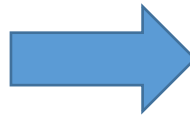
def main_loop(window, display_func):
    while not glfw.window_should_close(window):
        display_func(window)
        glfw.swap_buffers(window)
        glfw.poll_events()

    glfw.destroy_window(window)
    glfw.terminate()
#####
```

우리 프로그램 시작 부분과 새로운 함수

```
import glfw
from myglfw import *
from OpenGL.GL import *

def drawAxes():
    glBegin(GL_LINES)
    # x축 (0,0,0) - (1,0,0)
    glColor3f(1, 0, 0) # 빨간색
    glVertex3f(0,0,0)
    glVertex3f(1,0,0)
    # y축
    glColor3f(0, 1, 0) # 녹색
    glVertex3f(0,0,0)
    glVertex3f(0,1,0)
    # z축
    glColor3f(0, 0, 1) # 파란색
    glVertex3f(0,0,0)
    glVertex3f(0,0,1)
    glEnd()
```



OpenGL 프리미티브를 이용하여
x, y, z 축을 그리는 함수

그리기 코드

```
def initialize(window):
    glClearColor(0.0, 0.0, 0.0, 1.0)

def display(window):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

    glColor3f(0, 1, 1)
    glBegin(GL_TRIANGLES)
    glVertex3f(-0.5, -0.5, 0)
    glVertex3f(0.5, -0.5, 0)
    glVertex3f(0, 0.5, 0)
    glEnd()

    drawAxes()

def reshape(window, width, height):
    glViewport(0, 0, width, height)
```

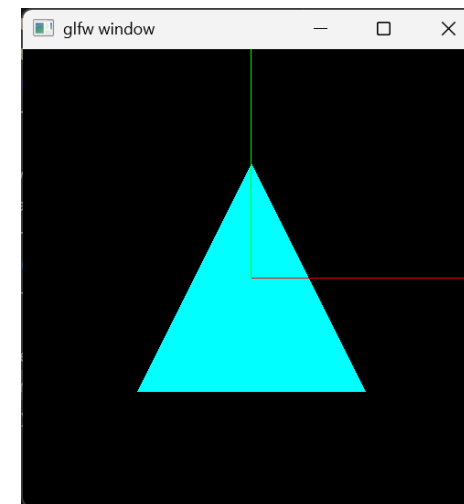
```
def main():
    window = initialize_window(500, 500, "glfw window")

    # Set up callbacks similar to GLUT
    glfw.set_framebuffer_size_callback(window, reshape)

    # Call initialize
    initialize(window)

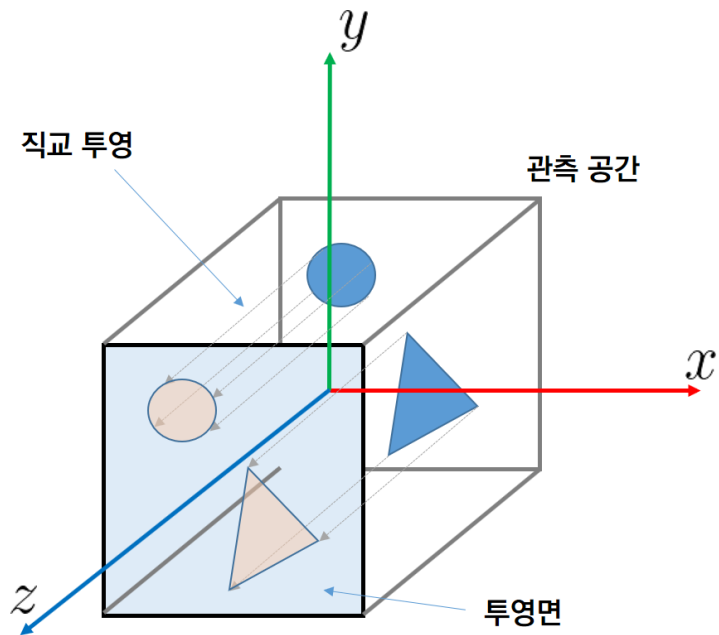
    # Main loop calling display
    main_loop(window, display)

if __name__ == "__main__":
    main()
```



카메라 모델

- 직교 투영 공간



$$f : \mathbf{V} \rightarrow \mathbf{V}$$

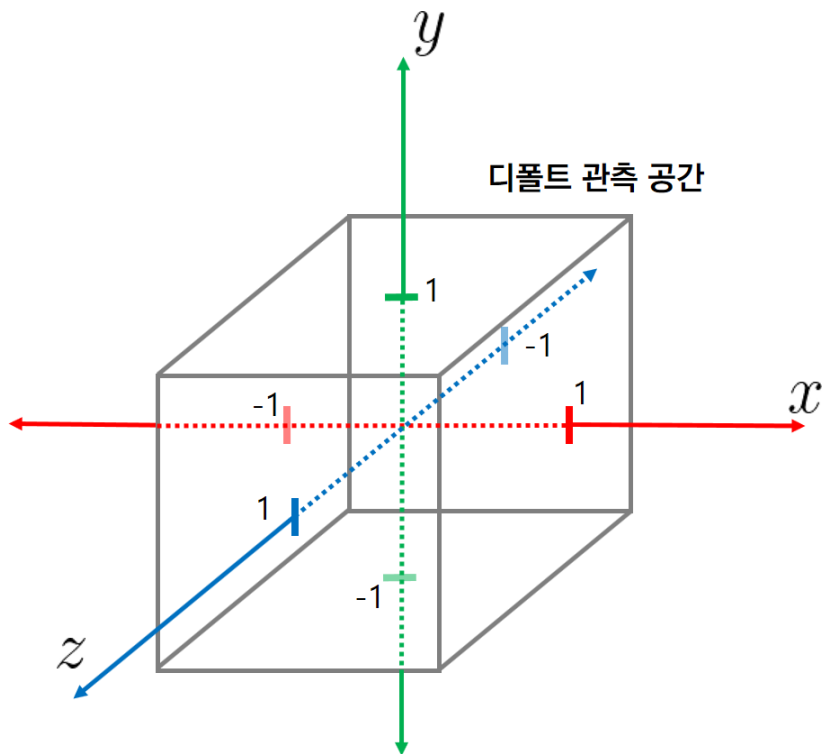
$$f(x, y, z) = (x, y, z_{proj})$$

이게 다 뭘 소리냐...

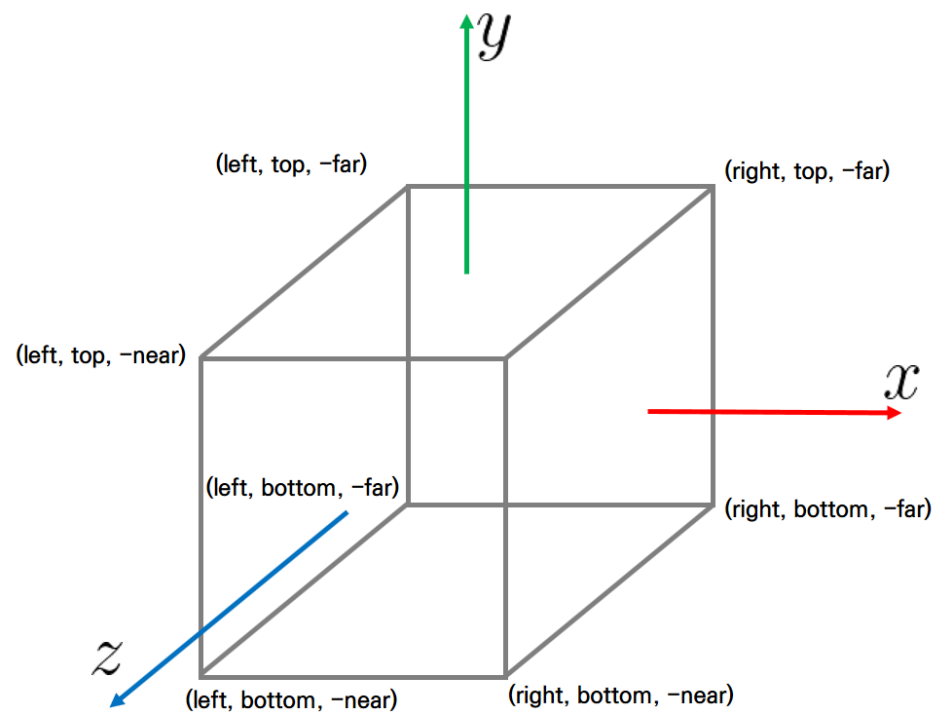
x, y 의 좌표는 그대로 유지
 z 만 그리는 면에 떨어트림

카메라 모델

- 디폴트 직교 투영 공간과 변경



```
glOrtho(left, right, bottom, top, near, far)
```



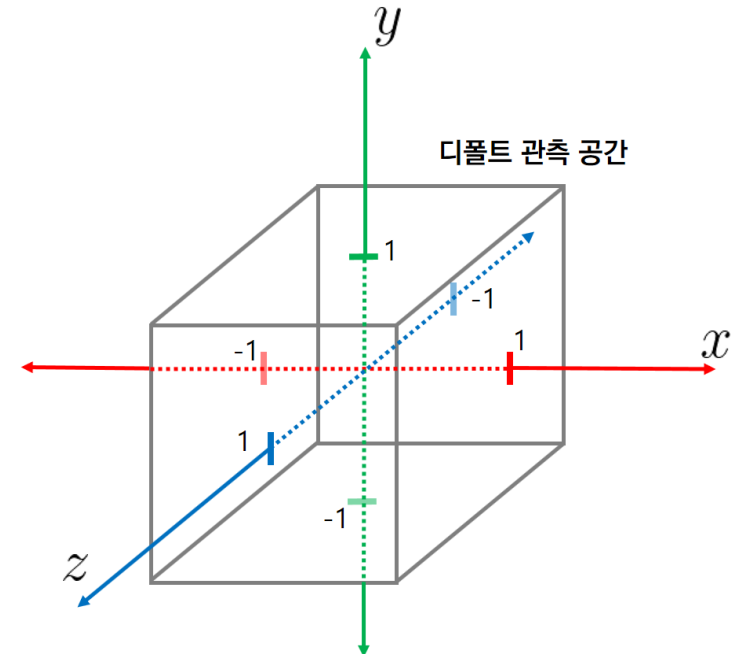
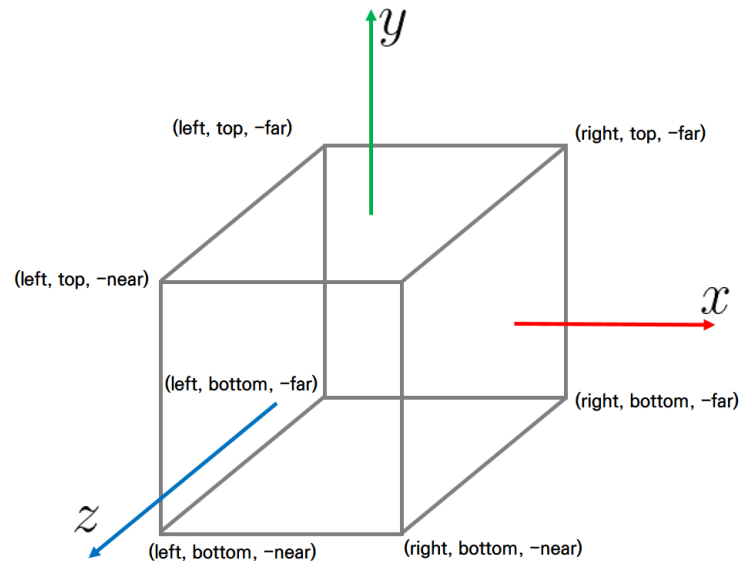
화면에 그리기 위해 좌표는 어떻게 바뀌나

- 정규 관측 공간으로 옮기기
 - 정규 관측 공간
 - x, y, z 축 방향으로 [-1, 1] 사이의 공간
 - 좌표를 정규 관측 공간으로 옮긴다는 것 →

$$x: [\text{left}, \text{right}] \Rightarrow x': [-1, 1]$$

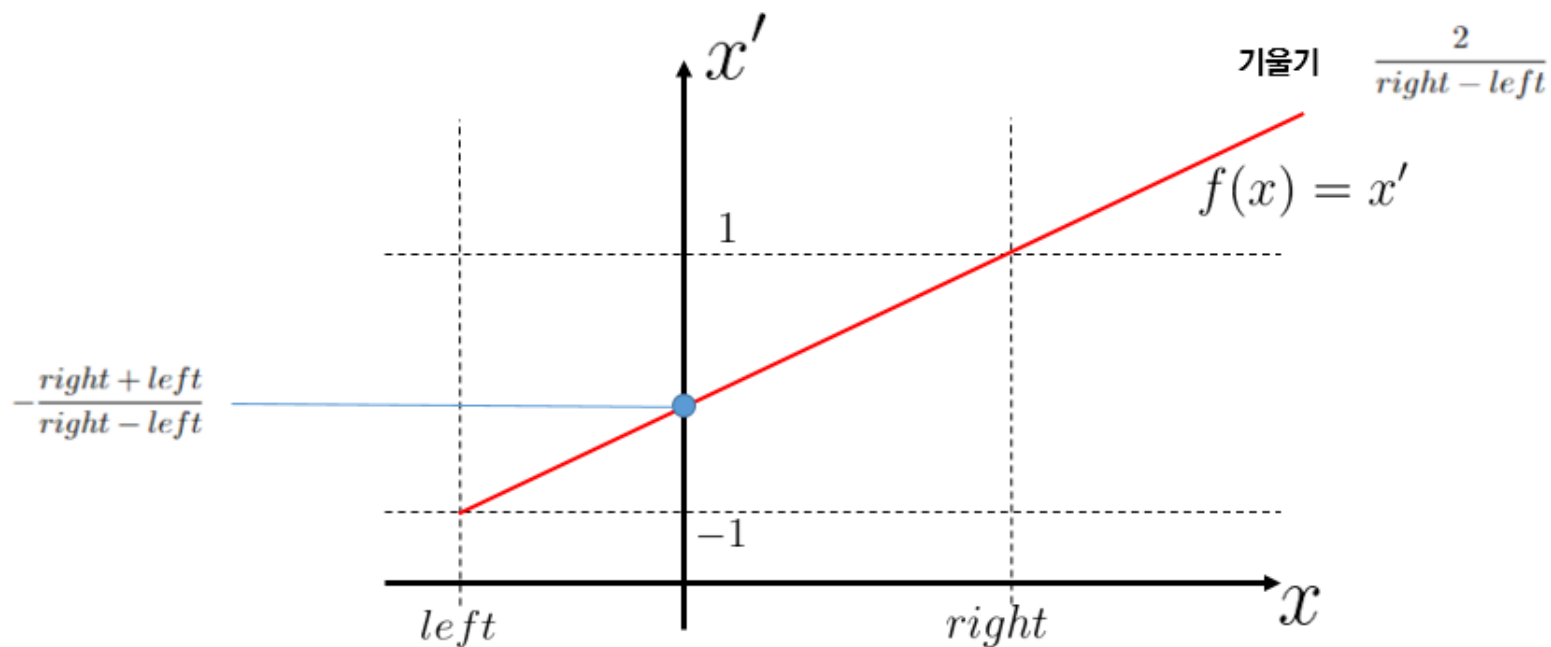
$$y: [\text{bottom}, \text{top}] \Rightarrow y': [-1, 1]$$

$$z: [\text{near}, \text{far}] \Rightarrow z': [-1, 1]$$



X 좌표를 바꾸어 보자

- left에 해당하는 값은 -1 , right에 해당하는 값은 1
- $right-left$ 의 간격은 길이 2의 간격으로 변환



X 좌표를 바꾸어 보자

- left에 해당하는 값은 -1, right에 해당하는 값은 1
- right-left의 간격은 길이 2의 간격으로 변환

$$x' = f(x) = \left(\frac{2}{right - left} \right) x - \frac{right + left}{right - left}$$

x,y,z 좌표를 바꾸어 보자

$$x' = f(x) = \left(\frac{2}{right - left} \right) x - \frac{right + left}{right - left}$$

$$y' = \left(\frac{2}{top - bottom} \right) y - \frac{top + bottom}{top - bottom}$$

$$z' = - \left(\frac{2}{far - near} \right) z - \frac{far + near}{far - near}$$



부호 바뀜 주의

직교 투영 = 선형 변환 행렬을 적용하는 것

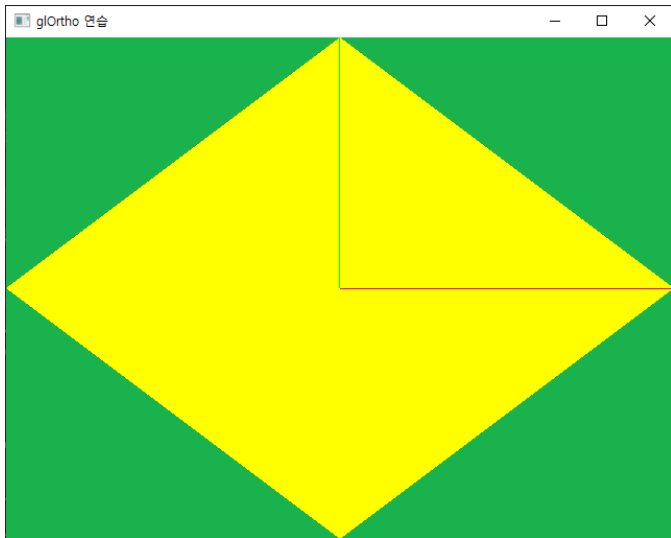
$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{2}{right-left} & 0 & 0 & -\frac{right+left}{right-left} \\ 0 & \frac{2}{top-bottom} & 0 & -\frac{top+bottom}{top-bottom} \\ 0 & 0 & \frac{-2}{far-near} & -\frac{far+near}{far-near} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$



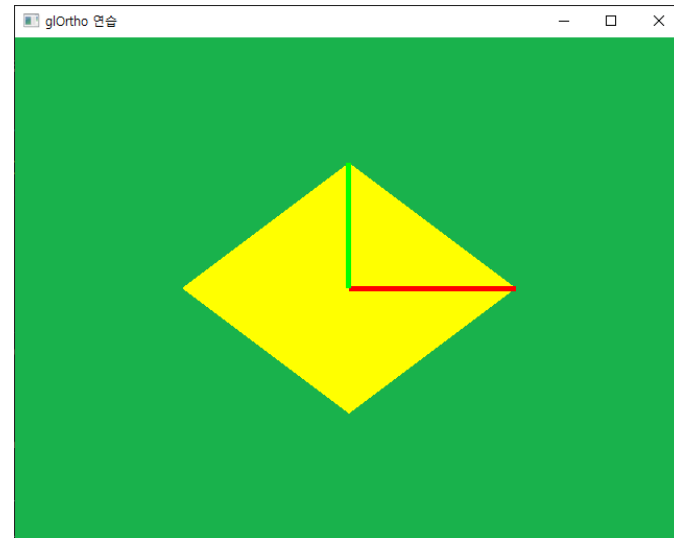
투영행렬: 모든 점들은 이 행렬이 곱해져서 최종 좌표가 결정된다

glOrtho 사용하기

```
def reshape(window, width, height):  
    glViewport(0, 0, width, height)  
    glMatrixMode(GL_PROJECTION)  
    glLoadIdentity()  
    glOrtho(-2, 2, -2, 2, -1, 1)
```

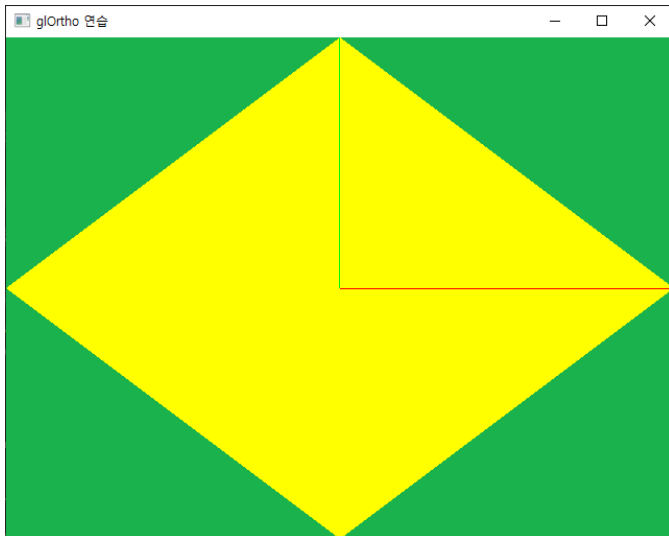


이렇게 안 됨...

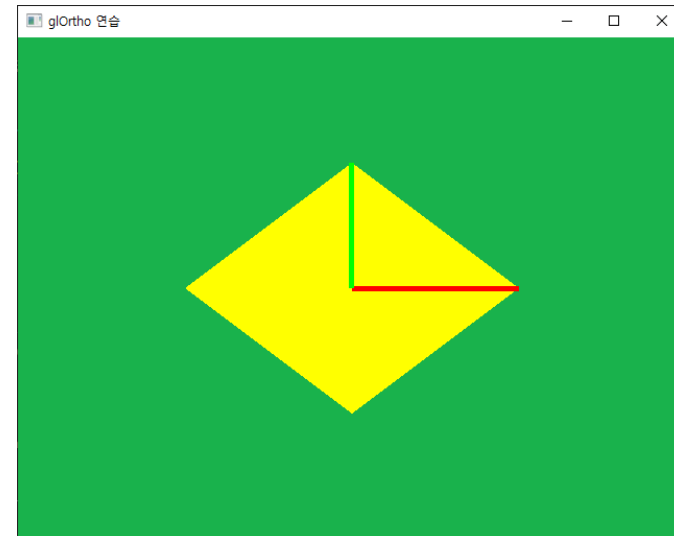


Reshape 함수를 강제로 한 번 호출해야 함

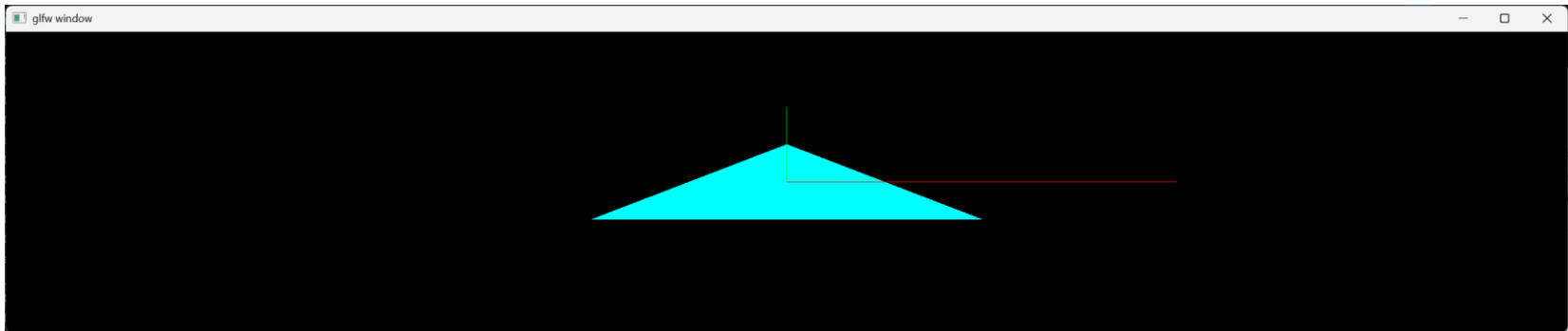
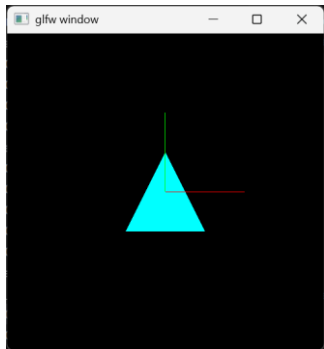
```
def main():  
    window = initialize_window(500, 500, "glfw window")  
  
    # Set up callbacks similar to GLUT  
    glfw.set_framebuffer_size_callback(window, reshape)  
  
    # Manually apply initial reshape  
    width, height = glfw.get_framebuffer_size(window)  
    reshape(window, width, height)
```



OK



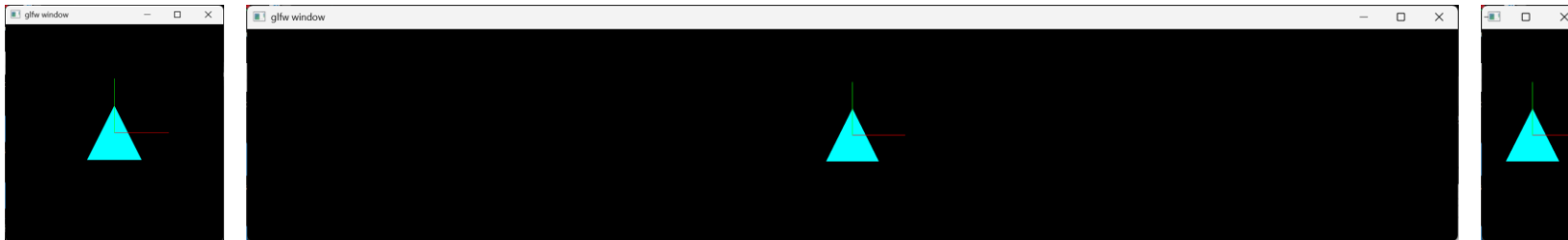
문제점



왜곡이 없는 투영

- 화면의 크기에 맞춰 투영행렬 설정

```
def reshape(window, width, height):  
    glViewport(0, 0, width, height)  
    aspRatio = width / height # 종횡비를 계산한다.  
    range = 2.0  
    glMatrixMode(GL_PROJECTION)  
    glLoadIdentity()  
    glOrtho(-range*aspRatio, range*aspRatio, -range, range, -1, 1)
```



투영의 관찰을 위한 두 개의 뷰포트

```
def display(window):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # 전체 창 크기 가져오기
    width, height = glfw.get_framebuffer_size(window)

    # 첫 번째 뷰포트: 왼쪽 절반, 기본 카메라 (앞에서 바라봄)
    glViewport(0, 0, width // 2, height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    asp_ratio = (width // 2) / height
    range_val = 2.0
    glOrtho(-range_val * asp_ratio, range_val * asp_ratio, -range_val, range_val, -10, 10) # z 범위 확장
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    gluLookAt(0, 0, 3, # eye position
              0, 0, 0, # look at
              0, 1, 0) # up vector

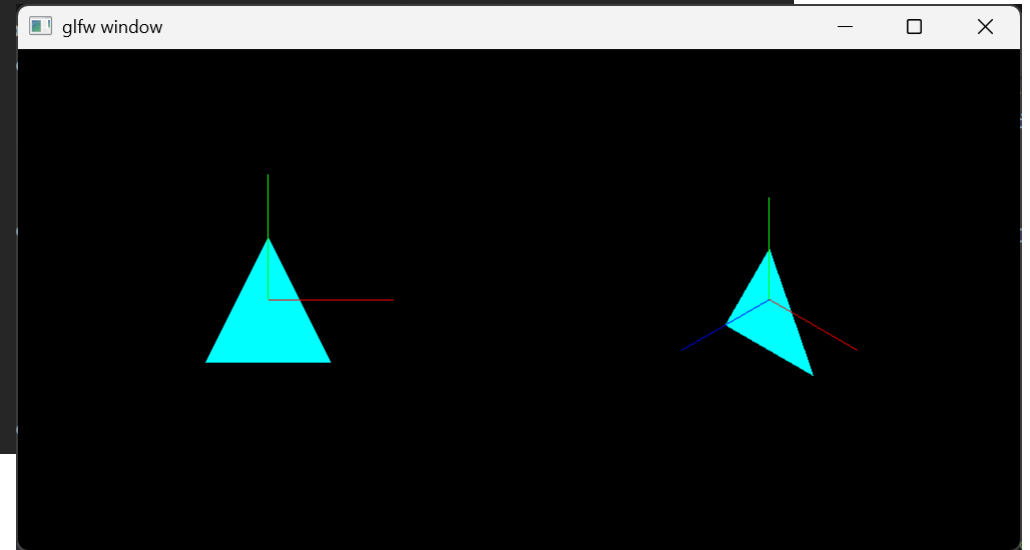
    myScene()

    # 두 번째 뷰포트: 오른쪽 절반, 다른 카메라 (측면에서 비스듬히 바라봄)
    glViewport(width // 2, 0, width // 2, height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    asp_ratio = (width // 2) / height
    glOrtho(-range_val * asp_ratio, range_val * asp_ratio, -range_val, range_val, -10, 10) # 동일한 프로젝션
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    gluLookAt(2, 2, 2, # eye position (다른 위치)
              0, 0, 0, # look at
              0, 1, 0) # up vector

    myScene()
```


공유된 장면

```
def myScene():  
    glColor3f(0, 1, 1)  
    glBegin(GL_TRIANGLES)  
    glVertex3f(-0.5, -0.5, 0)  
    glVertex3f(0.5, -0.5, 0)  
    glVertex3f(0, 0.5, 0)  
    glEnd()  
  
    drawAxes()
```



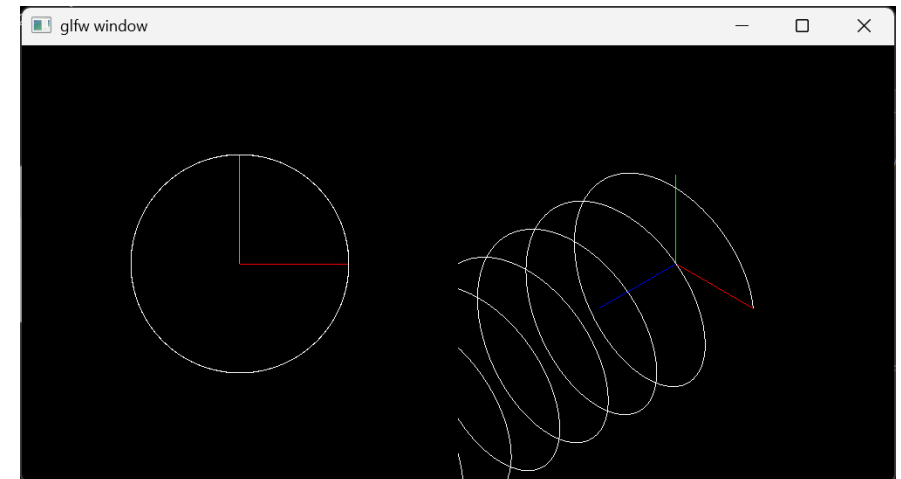
```
def reshape(window, width, height):  
    # reshape는 더 이상 전체 프로젝션을 설정하지 않음; display에서 처리  
    pass # 필요 시 다른 로직 추가 가능
```

두 뷰포트에 나선 그리기

```
def drawAxes():
    glBegin(GL_LINES)
    glColor3f(1,0,0) # red x axis
    glVertex3f(0,0,0); glVertex3f(1,0,0)
    glColor3f(0,1,0) # green y axis
    glVertex3f(0,0,0); glVertex3f(0,1,0)
    glColor3f(0,0,1) # blue z axis
    glVertex3f(0,0,0); glVertex3f(0,0,1)
    glEnd()

def drawHelix():
    glColor3f(1,1,1)
    glBegin(GL_LINE_STRIP)
    for i in range(1000):
        angle = i/10
        x, y = math.cos(angle), math.sin(angle)
        glVertex3f(x, y, angle/10)
    glEnd()
```

```
def myScene():
    drawAxes()
    drawHelix()
```



glOrtho 영역 확인하기

```
def drawBox(l, r, b, t, n, f): # glOrtho가 만드는 공간(육면체)을 가시화
    glColor3f(1, 1, 1)
    glBegin(GL_LINE_LOOP)
    # 앞면
    glVertex3f(l,t,n)
    glVertex3f(l,b,n)
    glVertex3f(r,b,n)
    glVertex3f(r,t,n)
    glEnd()

    glBegin(GL_LINE_LOOP)
    # 뒷면
    glVertex3f(l,t,f)
    glVertex3f(l,b,f)
    glVertex3f(r,b,f)
    glVertex3f(r,t,f)
    glEnd()
```

```
ortho_range = [l, r, b, t, n, f] = [-2, 2, -2, 2, -2, 2]
```

```
# 첫 번째 뷰포트: 왼쪽 절반, 기본 카메라 (앞에서 바라봄)
```

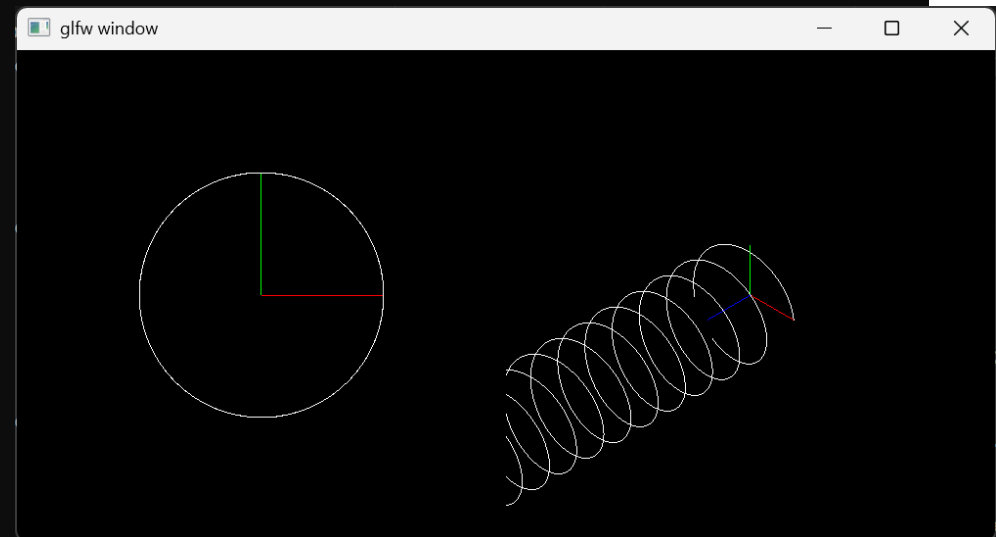
```
glViewport(0, 0, width // 2, height)
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
glOrtho(l, r, b, t, n, f) # z 범위 확장
glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
```

```
myScene()
```

```
# 두 번째 뷰포트: 오른쪽 절반, 다른 카메라 (측면에서 비스듬히 바라봄)
```

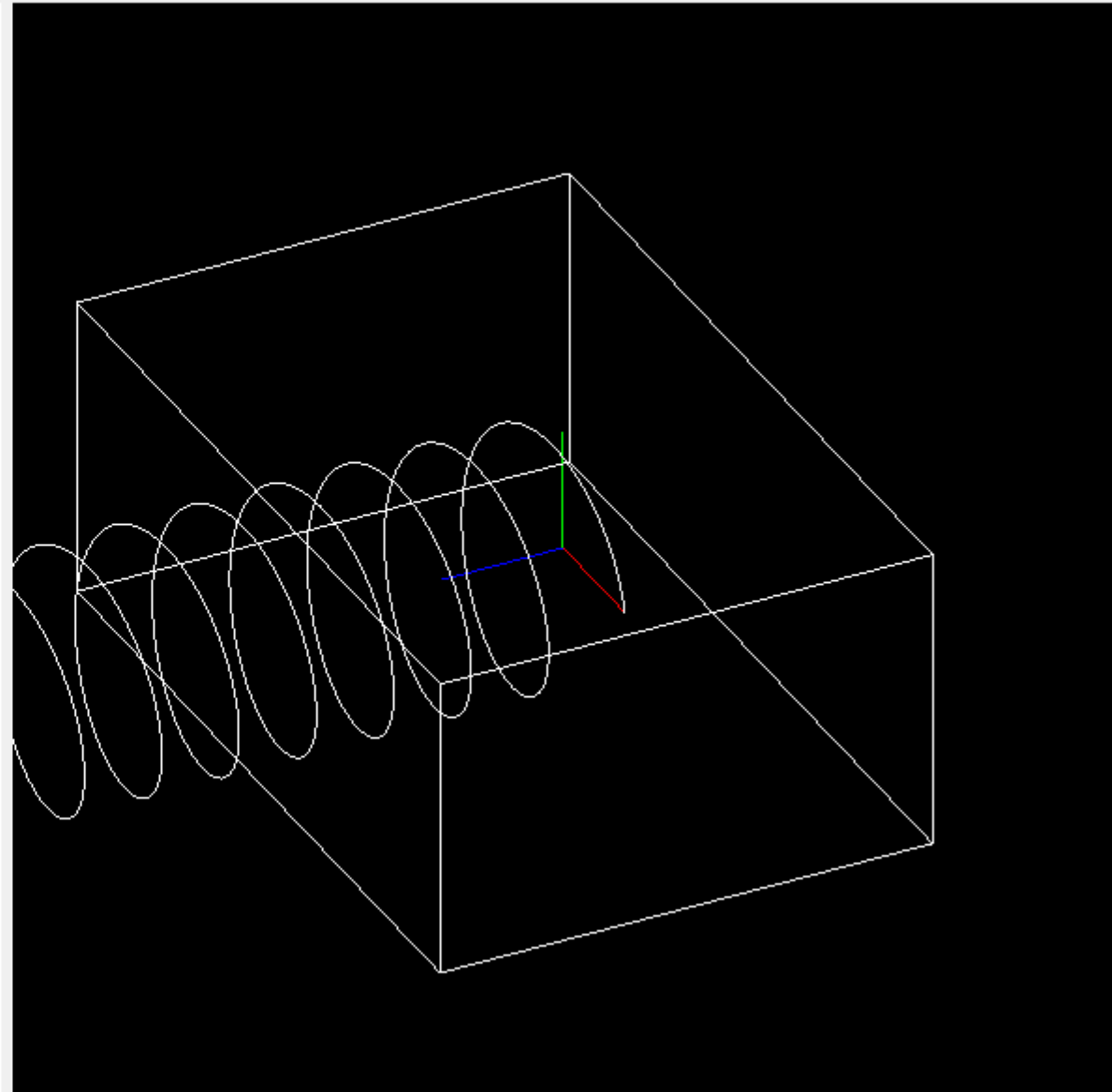
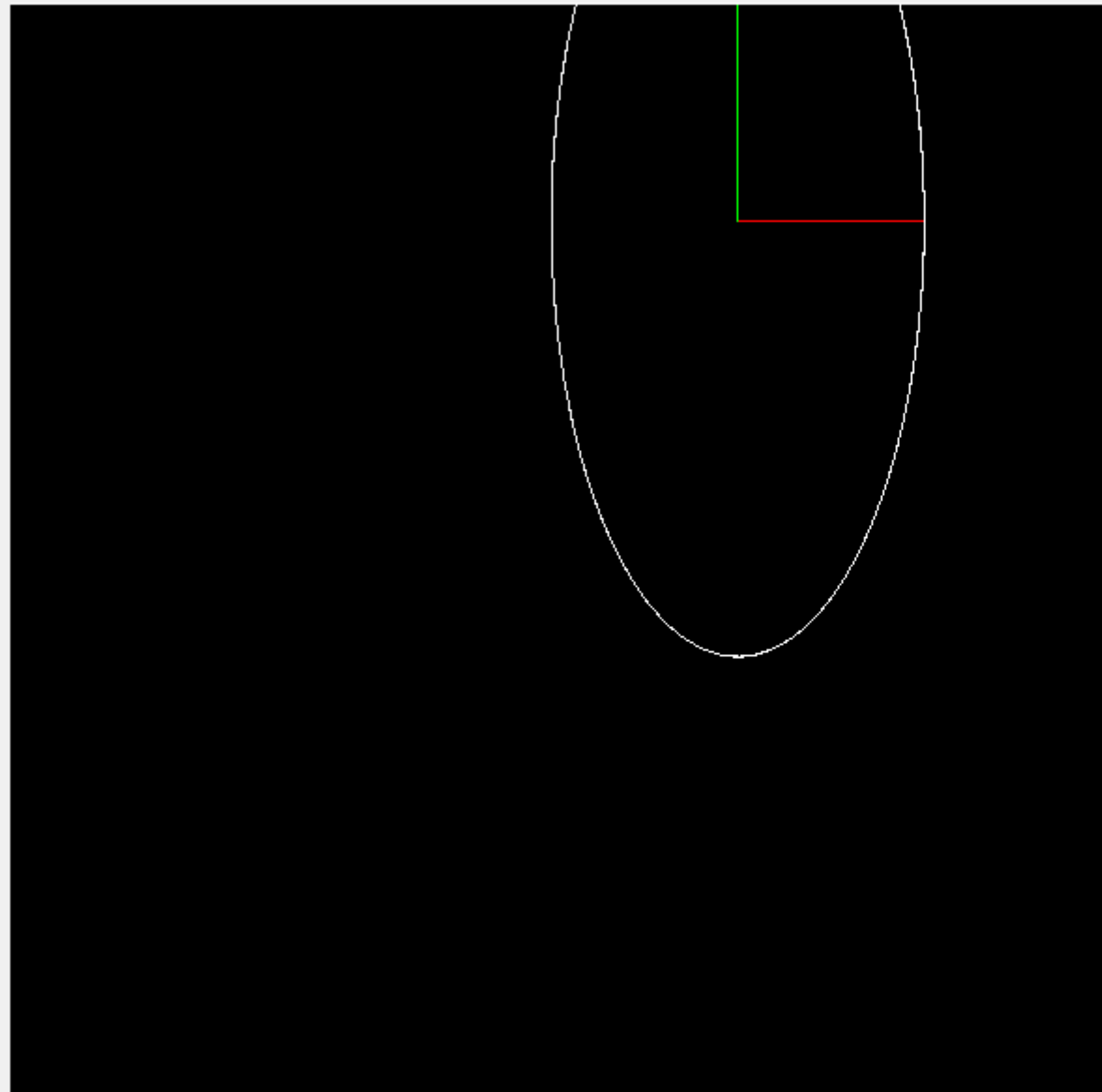
```
glViewport(width // 2, 0, width // 2, height)
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
glOrtho(l*2, r*2, b*2, t*2, n*2, f*2) # z 범위 확장
glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
gluLookAt(2, 2, 2, # eye position (다른 위치)
          0, 0, 0, # look at
          0, 1, 0) # up vector
```

```
myScene()
```



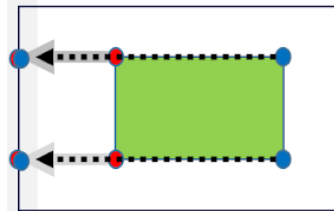
키보드를 통한 관측 공간 제어

```
def myScene():  
    drawAxes()  
    drawHelix()  
    drawBox(l, r, b, t, n, f)  
  
l, r, b, t, n, f = -2, 2, -2, 2, -2, 2  
  
def key_callback(window, key, scancode, action, mods):  
    global l, r, b, t, n, f  
    if action == glfw.PRESS:  
        if glfw.KEY_A == key :  
            l -= 0.1
```



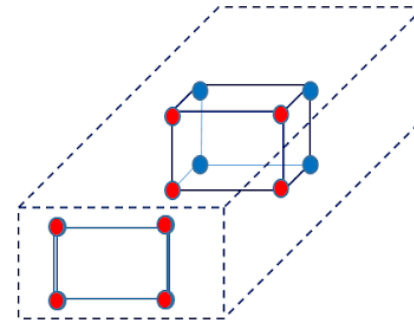
원근이 파악되는 투영 - glFrustum

직교 투영 (2차원)

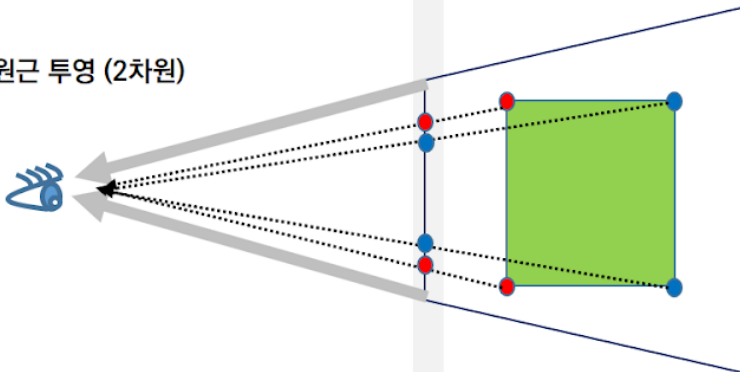


상이 맺히는 곳

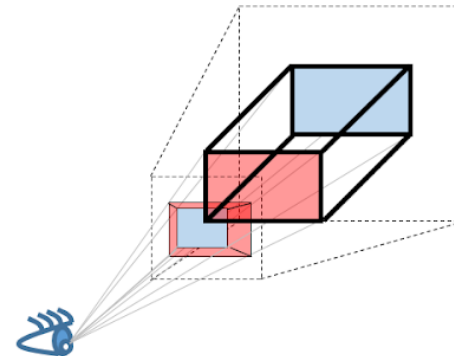
직교 투영 (3차원)



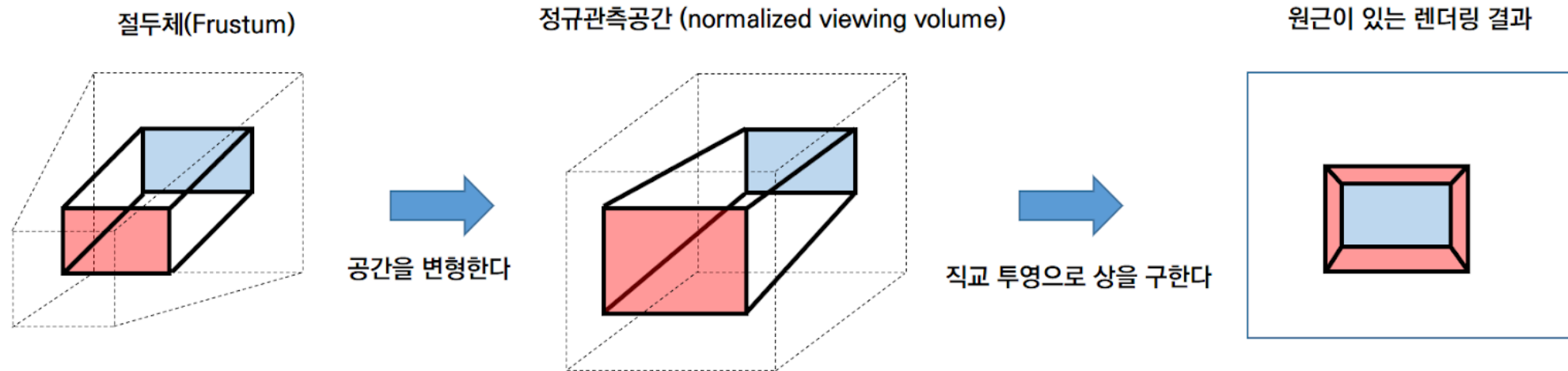
원근 투영 (2차원)



원근 투영 (3차원)

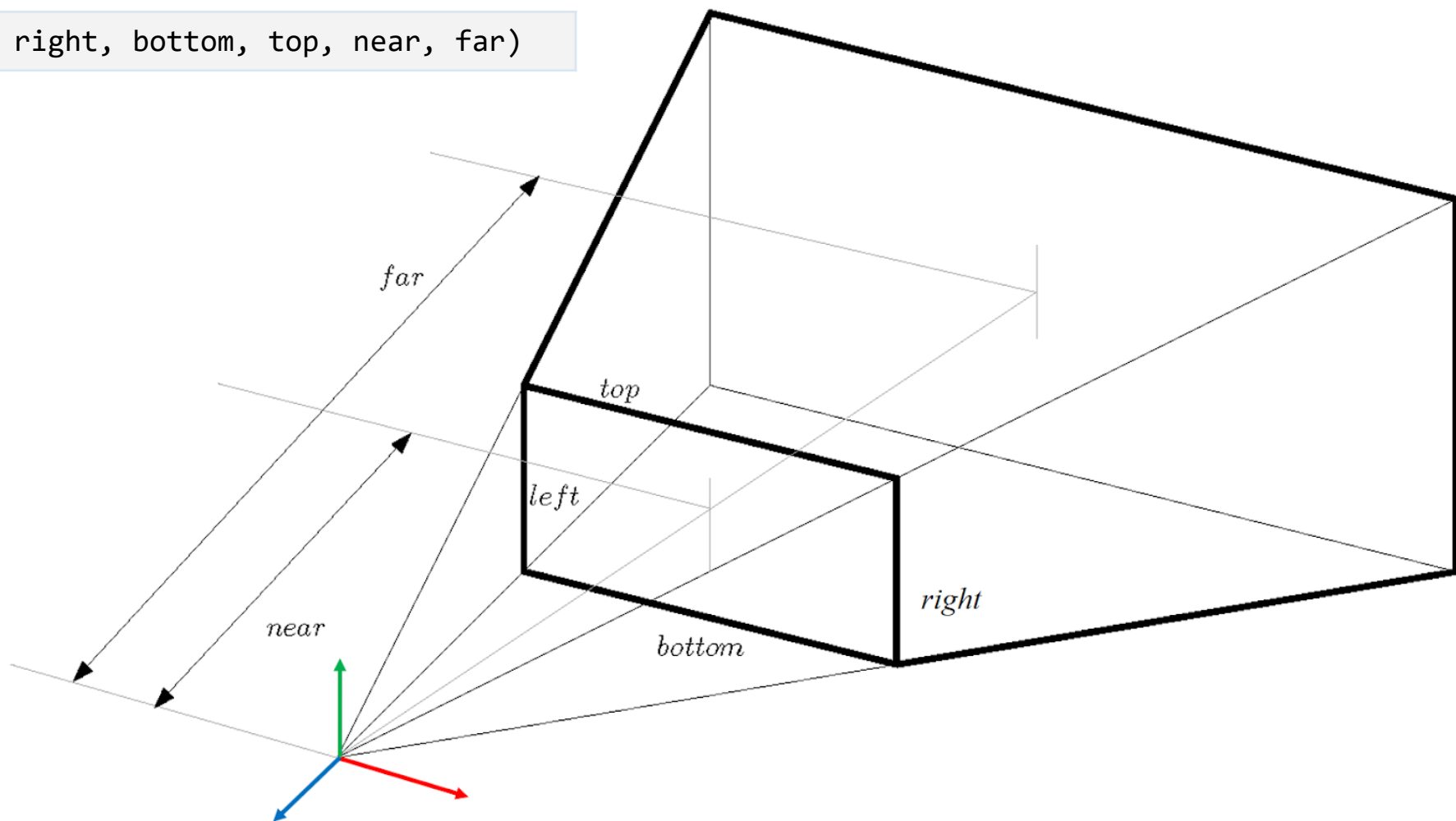


원근이 파악되는 투영 - glFrustum



원근이 파악되는 투영 - glFrustum

```
glFrustum(left, right, bottom, top, near, far)
```

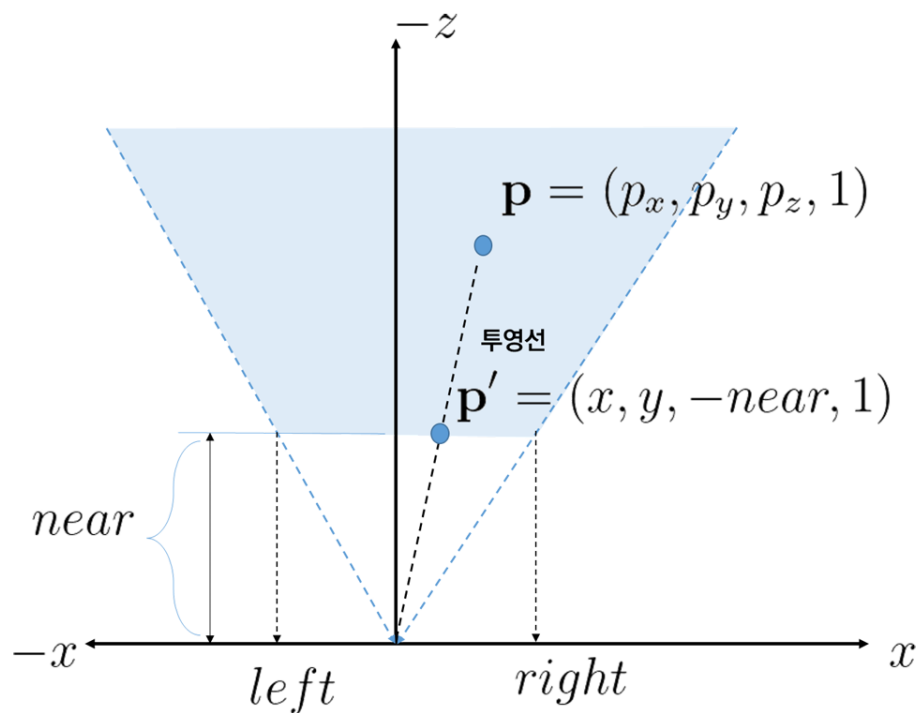


사용 방법

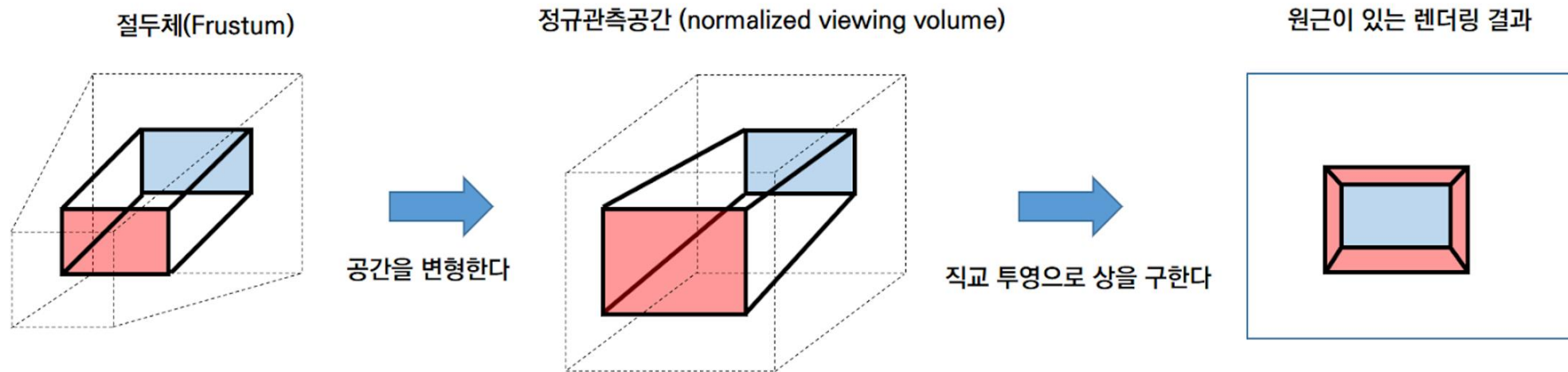
```
glMatrixMode(GL_PROJECTION)
glLoadIdentity();
glFrustum(l, r, b, t, n, f)
```

투영 행렬 수정 모드로 지정
투영 행렬을 항등행렬 **I**로 지정
현재 투영 행렬에 새로운 행렬을 곱한다.

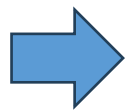
$$x = -near \frac{p_x}{p_z}$$



정규 좌표계로 이동



$$x = -near \frac{p_x}{p_z}$$



$$x' = \left(-\frac{2near}{right - left} \right) \frac{p_x}{p_z} - \frac{right + left}{right - left}$$

투영행렬

$$x' = \left(-\frac{2near}{right-left} \right) \frac{p_x}{p_z} - \frac{right+left}{right-left}$$

$$y' = \left(-\frac{2near}{top-bottom} \right) \frac{p_y}{p_z} - \frac{top+bottom}{top-bottom}$$

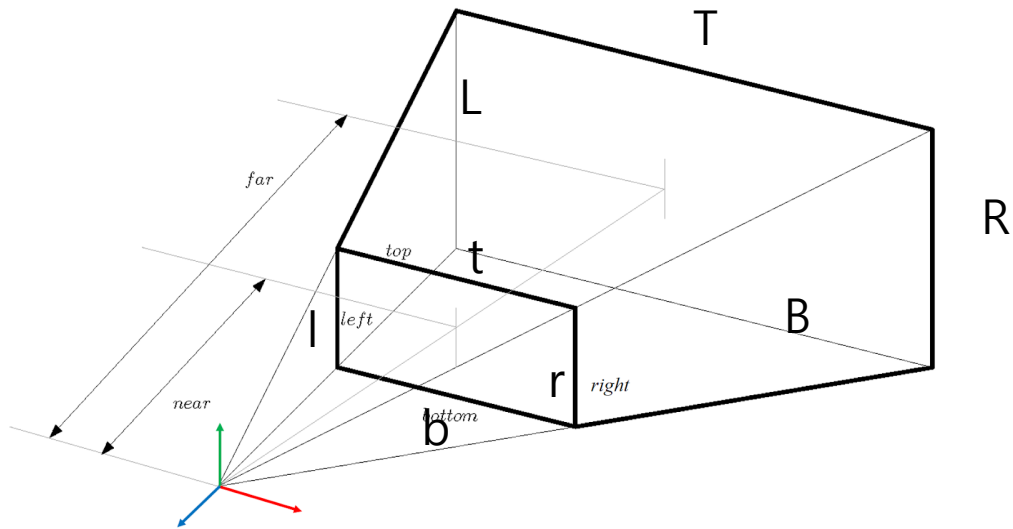
$$\mathbf{M} = \begin{pmatrix} \frac{2near}{right-left} & 0 & \frac{right+left}{right-left} & 0 \\ 0 & \frac{2near}{top-bottom} & \frac{top+bottom}{top-bottom} & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & \frac{-2near \cdot far}{far-near} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

* 상세한 유도 방법은 교재 문서 참고

절두체 관찰하기

```
glFrustum(l, r, b, t, n, f)
```

$$L = \frac{l}{n}f, \quad R = \frac{r}{n}f, \quad B = \frac{b}{n}f, \quad T = \frac{t}{n}f$$



```
def drawFrustum(l, r, b, t, n, f):  
    L = l * (f/n)  
    R = r * (f/n)  
    B = b * (f/n)  
    T = t * (f/n)  
    glColor3f(1, 1, 1)  
    glBegin(GL_LINE_LOOP)  
    # 앞면  
    glVertex3f(l,t,-n)  
    glVertex3f(l,b,-n)  
    glVertex3f(r,b,-n)  
    glVertex3f(r,t,-n)  
    glEnd()  
    glBegin(GL_LINE_LOOP)  
    # 뒷면  
    glVertex3f(L,T,-f)  
    glVertex3f(L,B,-f)  
    glVertex3f(R,B,-f)  
    glVertex3f(R,T,-f)  
    glEnd()  
  
    # 모서리 연결선 4개  
    glBegin(GL_LINES)  
    glVertex3f(l,t,-n)  
    glVertex3f(L,T,-f)  
    glVertex3f(l,b,-n)  
    glVertex3f(L,B,-f)  
    glVertex3f(r,b,-n)  
    glVertex3f(R,B,-f)  
    glVertex3f(r,t,-n)  
    glVertex3f(R,T,-f)  
    glEnd()
```

절두체 관찰하기

`l, r, b, t, n, f = -1, 1, -1, 1, 1.5, 5.5`

```
def display(window):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # 전체 창 크기 가져오기
    width, height = glfw.get_framebuffer_size(window)

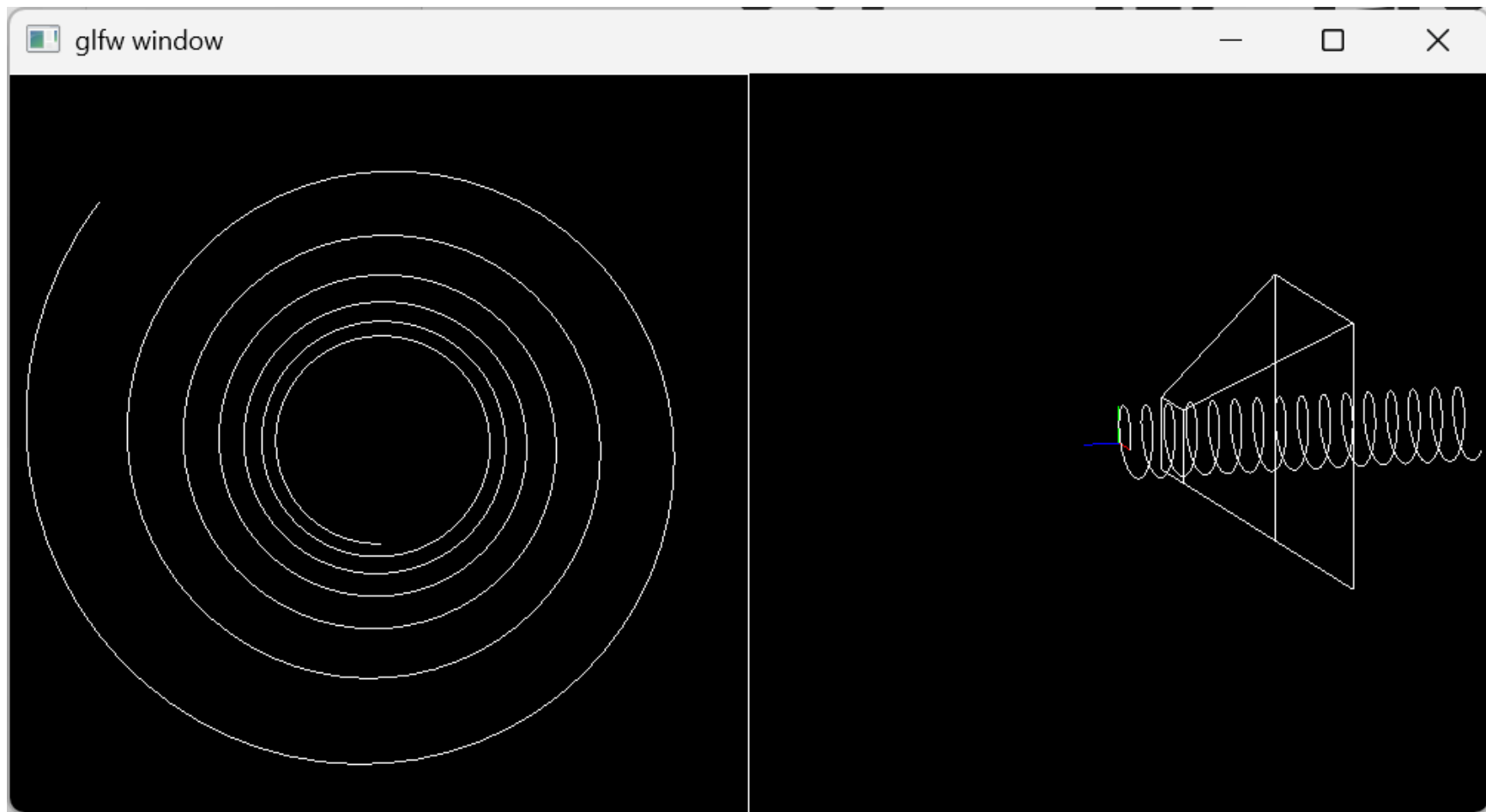
    # 첫 번째 뷰포트: 왼쪽 절반, 기본 카메라 (앞에서 바라봄)
    glViewport(0, 0, width // 2, height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    glFrustum(l, r, b, t, n, f) # z 범위 확장
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

    myScene()

    # 두 번째 뷰포트: 오른쪽 절반, 다른 카메라 (측면에서 비스듬히 바라봄)
    glViewport(width // 2, 0, width // 2, height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    glOrtho(-10, 10, -10, 10, -100, 100) # z 범위 확장
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    gluLookAt(50, 10, 15, # eye position (다른 위치)
             0, 0, 0, # look at
             0, 1, 0) # up vector

    myScene()
```

절두체 관찰하기



gluPerspective

